

Speed

SPRFST is built to start instantly and stay small. This chapter is about seeing where the time goes and what the compiler does about it.

Measuring a run

```
sprfst run main.spf --time
```

prints a line like

```
3 files  parse 1.2ms  check 0.8ms  lower 0.4ms  run 12.6ms  410233 instr  512 KB peak  2 gc
```

Read it from the left: how long the source took to read, to check, to turn into instructions, and then how long your program actually ran, how many instructions it executed, the high water mark of memory, and how many times the collector ran. If `parse` and `check` dominate, the program is small and the figure to care about is startup. If `run` dominates, carry on reading.

Timing part of a program

`time.clock()` is a monotonic clock in seconds. Wrap the part you are suspicious of:

```
use std.io
use std.time

fn slow_text(n: Int) -> Text {
  var out = ""
  for i in 0..n { out = out + "{i}," }
  give out
}

fn quick_text(n: Int) -> Text {
  var parts: [Text] = []
  for i in 0..n { parts.push("{i}") }
  give parts.join(",")
}

fn main() {
  let n = 4000

  let a = time.clock()
  let one = slow_text(n)
  let b = time.clock()
  let two = quick_text(n)
  let c = time.clock()

  io.say("growing text  {(b - a) * 1000.0} ms")
  io.say("list and join {(c - b) * 1000.0} ms")
  io.say("same length: {one.len() >= two.len()}")
}
```

```
}
```

Building text by repeated `+` copies everything it has so far each time. Collecting the pieces and joining once does the copy only once. That single habit is worth more than any compiler flag.

Benchmarks that live with the code

A `bench` block is timed instead of checked. The runner repeats it until the figure settles and reports operations per second.

```
use std.io

fn fib(n: Int) -> Int {
  if n < 2 { give n }
  give fib(n - 1) + fib(n - 2)
}

bench "fib 15" {
  let _ = fib(15)
}

bench "list of 1000" {
  var xs: [Int] = []
  for i in 0..1000 { xs.push(i) }
}

fn main() {
  io.say("fib(15) = {fib(15)}")
}
```

```
sprfst test
```

runs the tests first and then every benchmark.

What the optimiser does

Lowered code goes through five passes, in order:

1. **Constant folding** —arithmetic, comparisons and text joins between literals are worked out once, at compile time.
2. **Move removal** —registers copied onto themselves disappear.
3. **Dead code elimination** —a pure instruction whose result is never read is deleted. Anything that can have an effect is kept.
4. **Jump threading** —a jump to a jump becomes one jump.
5. **Compaction** —the holes left behind are closed up and every jump target is renumbered.

Levels pick how hard it tries:

Flag	Passes	Used by
------	--------	---------

-00	none	debugging, when you want instructions to match source
-01	one round	the default for sprfst run
-02	three rounds	the default for sprfst build
-03	three rounds, same as -O2 today	reserved for the native backend

You can watch it work:

```
sprfst build main.spf -02
```

reports the counts for each pass, and `--ir` writes the instructions to `build/program.spir` where you can read them:

```
built 2 functions, 20 instructions -> ./build/program.spir
opt   folded 2, moves 0, dead 0, jumps 0, nops 0
```

The listing is plain text, one instruction per line, with the source line it came from in the margin. It is the same listing the debugger steps through.

Where the time really goes

A few things that matter more than anything else, roughly in order:

Text built in a loop. Use a list and `join`, as above.

Work inside a loop that could be outside it. Hoist the constant parts out by hand; the optimiser will not move a function call for you because it cannot know the call is pure.

Maps where a list would do. A map lookup hashes; indexing a list does not.

Growing a list one element at a time is fine —it doubles its capacity, so a million pushes is a handful of allocations.

`clone` on a large structure. **It copies. Pass the reference unless** you need an independent copy.

The collector runs between statements and only walks live data. Short lived values are cheap; holding on to everything is not.

Memory

`sys.memory()` gives the bytes the runtime is holding right now, which is useful inside a program:

```
use std.io
use std.sys

fn main() {
  let before = sys.memory()
  var big: [Int] = []
  for i in 0..200000 { big.push(i) }
```

```
let after = sys.memory()
io.say("{big.len()} numbers cost about {(after - before) / 1024} KB")
}
```

A word about the backend

Today `sprfst run` executes the lowered instructions directly in a register machine written in C: no process launch, no JIT warm up, a few milliseconds from typing the command to your first line of output. `sprfst build` writes that same program out as a readable listing rather than a standalone executable. A native code generator is the next thing on the list, and the `-O3` level is reserved for it.

Exercise

Write a benchmark that compares looking a word up in a `Map<Text, Int>` against scanning a `[Text]` with `index_of`, for 10, 100 and 1000 entries, and find the size where the map starts to win.